

Задача 3. Архитектурное улучшение учебной информационной системы

1. Постановка задачи

В учебной информационной системе правила расчета скидки, проверки права на льготы и часть логики обработки заявок реализованы прямо в контроллерах и сервисах, которые напрямую завязаны на HTTP-слой, ORM, SQL-запросы и конкретные технические механизмы хранения данных. При изменении схемы базы данных или внешнего API команде приходится менять не только технический код, но и переписывать участки бизнес-логики. Поддержка и тестирование системы становятся все сложнее.

Предположим, что система будет дальше развиваться, требования к ней будут меняться, а команда хочет уменьшить зависимость бизнес-правил от инфраструктурных деталей.

Задание: укажите, что в такой структуре архитектурно неудачно, какой подход из курса здесь уместен и за счет чего он решает проблему. Достаточно концептуального ответа без детального описания классов и интерфейсов.

2. Краткий анализ исходной системы

В текущей структуре правила учебной системы реализованы рядом с техническим кодом. Контроллеры и сервисы одновременно принимают HTTP-запросы, работают с ORM и SQL, обращаются к внешним API и выполняют предметные вычисления.

Архитектурная проблема: бизнес-логика зависит от инфраструктуры. Расчет скидок, проверка права на льготы и обработка заявок оказываются привязаны к конкретной БД, ORM, структуре таблиц, HTTP-слою и формату внешних сервисов.

Дополнительно нарушаются разделение ответственности и инверсия зависимостей: контроллеры и технические сервисы начинают принимать предметные решения, а бизнес-правила зависят от внешних механизмов, хотя должно быть наоборот.

Основные последствия:

- контроллеры и сервисы получают слишком много обязанностей;
- предметные правила сложно тестировать без запуска БД, HTTP-сервера и внешних сервисов;
- технические изменения плохо локализуются;
- возрастает риск сломать расчет скидок, льгот или обработку заявок при изменении SQL, ORM или API;
- система хуже готова к изменению требований.

3. Подход к исправлению

Для такой ситуации основным подходом является Clean Architecture. Hexagonal Architecture / Ports and Adapters можно рассматривать как близкий способ реализовать ту же идею через порты и адаптеры. Главная цель — отделить бизнес-ядро от инфраструктурных деталей.

В бизнес-ядро выносятся правила предметной области: расчет скидок, проверка льгот, изменение статусов заявок и сценарии обработки заявок. HTTP, ORM, SQL, БД и внешние API не должны определять эти правила.

Контроллеры становятся тонкими: принять запрос, преобразовать входные данные, вызвать нужный сценарий и вернуть ответ. Расчеты и предметные решения должны находиться не в контроллерах, а в доменном или use case слое.

4. Целевая структура слоев

Слой	Роль	Что размещается
Domain / бизнес-ядро	Содержит правила предметной области.	Скидки, льготы, статусы заявок, доменные сущности, value objects, политики.
Use Cases / Application	Описывает сценарии работы системы.	Рассчитать скидку, проверить льготу, обработать заявку, применить правила в нужном порядке.
Ports / Interfaces	Фиксирует абстракции, нужные ядру.	Абстракции доступа к данным, внешнему сервису льгот, уведомлениям и текущему времени.
Interface Adapters	Связывает внешний мир с ядром.	Контроллеры, DTO, мапперы, presenters, входные и выходные адаптеры.
Infrastructure	Содержит технические детали.	БД, ORM, SQL, миграции, API-клиенты, кеши, очереди, конфигурация.

5. Правило зависимостей

Зависимости должны быть направлены внутрь: внешние слои зависят от бизнес-ядра, а бизнес-ядро не зависит от внешних слоев.

Внутренняя логика не должна импортировать контроллеры, ORM-сущности, SQL-клиенты, HTTP-клиенты, фреймворки и классы конкретных внешних API. Для связи используются абстракции, порты и инверсия зависимостей.

- use case описывает, какие данные и действия ему нужны;
- для доступа к данным, внешнему сервису льгот и уведомлениям объявляются абстракции;
- инфраструктура реализует эти абстракции через SQL, ORM или HTTP-клиент;
- контроллер вызывает use case, но не содержит бизнес-расчетов;
- при замене БД или внешнего API меняется адаптер, а не предметное правило.

6. Практическая схема перехода

1. Выделить бизнес-правила в domain/use case слой: расчет скидки, проверка льгот, сценарий обработки заявки.
2. Разгрузить контроллеры: оставить прием HTTP-запросов, DTO, коды ответов, базовую проверку формата и вызов сценария.
3. Разделить доменные модели, DTO и ORM-сущности, чтобы структура таблиц не попадала в бизнес-логику.
4. Ввести абстракции для хранения данных, проверки льгот, уведомлений и внешних интеграций.
5. Реализации абстракций вынести наружу: SQL/JPA-репозиторий, HTTP-клиент внешнего API, SMTP/SMS/Telegram-адаптер.
6. Добавить мапперы между DTO, ORM-моделями и доменными объектами.

7. Тестировать domain/use case слой отдельно; адаптеры БД и внешних API проверять интеграционными тестами.

7. Пример до и после

До: контроллер сам принимает HTTP-запрос, обращается к ORM или SQL, вызывает внешний API льгот и сразу внутри себя рассчитывает скидку.

После: контроллер только передает данные в сценарий “рассчитать стоимость обучения”, а сам расчет выполняется в бизнес-ядре. Доступ к БД и внешнему API скрыт за адаптерами.

8. Что не нужно делать

Проблему не решит простое переименование классов или перенос файлов по новым папкам. Если бизнес-логика продолжит напрямую зависеть от ORM, SQL, HTTP и внешних API, архитектурная связность останется прежней.

Ключевой критерий правильного решения — изменить направление зависимостей: бизнес-правила остаются внутри самостоятельного ядра, а технические механизмы подключаются снаружи через адаптеры.

9. Локализация изменений после разделения слоев

Изменение	Где меняется код	Что не меняется
Изменилась схема БД	Адаптер хранения, ORM-модель, SQL-запрос, миграция.	Правила расчета скидок, льгот и обработки заявок.
Заменяли ORM	Инфраструктурный репозиторий и маппинг данных.	Use case и доменные правила.
Изменился внешний API проверки льгот	API-клиент и адаптер внешнего сервиса льгот.	Сценарий проверки права на льготу.
Изменился формат HTTP-запроса	Контроллер, DTO, маппер входных данных.	Доменная модель и бизнес-правила.
Появилось новое правило скидки	Domain/use case слой.	HTTP-контроллер, SQL-запросы и внешний API, если их контракт не изменился.

10. Инструменты и технологии по слоям

Слой	Подходящие средства	Назначение
Domain	Чистый код языка; value objects; unit-тесты.	Хранить бизнес-правила без зависимости от фреймворков.
Use Cases / Ports	Application services; порты; dependency injection.	Описать сценарии и разорвать прямую зависимость ядра от БД, времени, API и уведомлений.
Web / Controllers	Spring MVC, Javalin, ASP.NET Web API, Gin, FastAPI; DTO; OpenAPI.	Принимать HTTP-запросы и передавать данные в use case.
Persistence / External Adapters	PostgreSQL; JPA/Hibernate, JDBC, jOOQ, MyBatis, GORM; HTTP-клиенты.	Реализовать хранение и интеграции, скрыв SQL, ORM и внешние API от бизнес-ядра.
Testing / Observability	JUnit/Mockito, Testcontainers, WireMock; логирование и метрики.	Тестировать ядро отдельно, адаптеры — интеграционно, ошибки — разбирать быстрее.

11. Ожидаемый результат

После перехода к такой структуре предметная логика становится стабильной частью системы. БД, ORM, HTTP-контроллеры и внешние сервисы становятся заменяемыми техническими деталями.

Это повышает тестируемость: расчет скидки, проверку льгот и обработку заявки можно проверить обычными unit-тестами без сервера, реальной БД и внешнего API. Интеграционные тесты остаются для адаптеров, где действительно проверяется работа с инфраструктурой.

Сопровождение также упрощается: изменение схемы таблиц, замена ORM или изменение внешнего API локализуются во внешнем слое и не требуют переписывать бизнес-правила.

12. Результат работы

Главный вывод: проблему нужно решать отделением бизнес-ядра от инфраструктуры, а не косметическим разбиением контроллеров. Основной подход — Clean Architecture; близкая практическая форма — Ports and Adapters. За счет инверсии зависимостей правила предметной области отделяются от HTTP, ORM, SQL, БД и внешних API, поэтому бизнес-логика становится устойчивее к изменениям, а система — проще для тестирования, сопровождения и развития.